

ECT* DTP-TALENT 2025:
Quantum computing for nuclear physics

Ryan LaRose

June 2025
Trento, Italy

Contents

1	Time evolution via Trotter	2
1.1	Problem definition	2
1.2	Trotter formulae	3
1.3	Proof of the first order formula	3
2	Quantum circuits for Trotter evolution	6
2.1	The general idea	6
2.2	Exponentials of individual Paulis	7
3	Coding exercise: Trotterized time evolution for the transverse-field Ising model	9
3.1	Overview	9
3.2	The transverse-field Ising model Hamiltonian	10
3.3	Exact time evolution	11
3.4	Trotterized time evolution	12
3.5	Further reading and exercises	15

Lecture

1

Time evolution via Trotter

Contents

1.1	Problem definition	2
1.2	Trotter formulae	3
1.2.1	The first order Trotter formula	3
1.2.2	Remarks and higher order formulae	3
1.3	Proof of the first order formula	3

1.1 Problem definition

Arguably the most fundamental task in quantum mechanics/information processing is time evolution under a Hamiltonian $\mathcal{H}$ given an initial state $|\psi(0)\rangle$. Schrödinger's equation

$$i\frac{d|\psi\rangle}{dt} = \mathcal{H}|\psi\rangle \quad (1.1)$$

tells us the solution (assuming time-independent $\mathcal{H}$) is

$$|\psi(t)\rangle = e^{-i\mathcal{H}t}|\psi(0)\rangle. \quad (1.2)$$

Typically we represent the Hamiltonian in the Pauli basis

$$\mathcal{H} = \sum_{i=1}^n c_i P_i. \quad (1.3)$$

Let us say that $\mathcal{H}$ acts on n qubits (i.e., is of dimension $2^n \times 2^n$) so that each Pauli string P_i has the form $\sigma_1 \otimes \dots \otimes \sigma_n$ where $\sigma \in \{I, X, Y, Z\}$. The c_i are of course (real-valued) coefficients.

If $[P_i, P_j] := P_i P_j - P_j P_i = 0$ for all i, j , we could evaluate (1.2) easily as the product of exponentials

$$e^{-i\mathcal{H}t} = e^{-ic_1 P_1 t} \dots e^{-ic_N P_N t}. \quad (1.4)$$

While all terms generally do not commute, the Trotter formula tells us this is still approximately true.

1.2 Trotter formulae

1.2.1 The first order Trotter formula

Theorem 1.1 (Trotter formula). If $\|A\| \leq 1$ and $\|B\| \leq 1$, then

$$\|e^{A+B} - e^A e^B\| = O(\|[A, B]\|). \quad (1.5)$$

Further, for $r = 1, 2, 3, \dots$,

$$\|e^{A+B} - [e^{A/r} e^{B/r}]^r\| = O(\|[A, B]\|/r). \quad (1.6)$$

The latter formula is extremely useful — it tells us that, at the cost of r Trotter steps, we can linearly decrease the error, and the limit $r \rightarrow \infty$ implies zero error. In this lecture, we're going to prove (1.5). In practice, we use (1.6).

1.2.2 Remarks and higher order formulae

Before the proof, there are several remarks and variations on the Trotter formula which are worth stating. First, we can re-insert time t via the mapping $A \mapsto At$ and $B \mapsto Bt$ in (1.6) to see that

$$\|e^{(A+B)t} - [e^{At/r} e^{Bt/r}]^r\| = O(\|[A, B]\|t^2/r). \quad (1.7)$$

You will typically see the statement of error for (first-order) Trotter is $O(t^2/r)$, since the assumptions $\|A\| \leq 1$ and $\|B\| \leq 1$ imply that $\|[A, B]\|$ is bounded and could be absorbed into the $O(\cdot)$. Here we will retain the commutator to stress its importance in our proof, and serve as a lower-level understanding of where the t^2 dependence comes from rather than just memorizing it.

Second, for more than two terms, i.e. $\exp\left(\sum_{i=1}^N H_i\right)$, the error term goes as $\sum_{i,j=1}^N \|[H_i, H_j]\|$. Note that this is the same form of the Hamiltonian (1.3) except we absorbed coefficients into Pauli terms: $H_i := c_i P_i$.

Third, it is worth remarking that there are higher order Trotter formulae, often called Lie-Suzuki-Trotter formulae. For example, the second order Trotter formula for $\mathcal{H} = A + B$ is $e^{At/2} e^{Bt} e^{At/2}$ and has an error which scales as t^3 . Thus for short times the second order formula is more accurate. The cost is a longer expression, translating to a longer quantum circuit. If quantum computers could implement gates with perfect fidelity, the second order Trotter formula would be preferable due to the lower error. However one has to balance algorithmic error with hardware error, at least on current noisy quantum computers. There are some cases in which terms in the second order formula can be combined and simplified to produce a quantum circuit with the same, or negligibly larger, depth than the first order formula [FICS24]. Last, we write the general case: for an N term Hamiltonian $\sum_{i=1}^N H_i$, the second order formula has the form $e^{H_1 t/2} \dots e^{H_N t/2} e^{H_N t/2} \dots e^{H_1 t/2}$.

Higher order formulas can be built recursively in the following manner [CST⁺21].

Theorem 1.2 (Higher order Trotter formulae). If we let $\delta_p(t)$ denote the p th order (Lie-Suzuki-)Trotter formula, then

$$\delta_{2k}(t) = \delta_{2k-2}^2(u_k t) \delta_{2k-2}([1 - 4u_k]t) \delta_{2k-2}^2(u_k t) \quad (1.8)$$

where $u_k := \frac{1}{4-4^{1/(2k-1)}}$ has an error which scales as t^{2k+1} .

1.3 Proof of the first order formula

Let us now prove the simplest version of the Trotter formula in Theorem 1.1.

Proof. We can Taylor expand

$$e^{A+B} = \sum_{j=0}^{\infty} (A+B)^j / j! = I + (A+B) + \frac{1}{2}(A^2 + AB + BA + B^2) + \dots \quad (1.9)$$

Similarly, we can Taylor expand

$$e^A e^B = \left(\sum_{j=0}^{\infty} A^j / j! \right) \left(\sum_{k=0}^{\infty} B^k / k! \right) = I + (A + B) + \frac{1}{2}(A^2 + 2AB + B^2) + \dots \quad (1.10)$$

Already the difference is apparent. Note that in (1.9) the quadratic term has a BA term that isn't present in the quadratic term in (1.10). Of course this is because in (1.10) all the A 's are on the left and all the B 's are on the right — let's say the terms are (alphabetically) *sorted*. However in (1.9) the terms are unsorted — there can be B 's on the left. We *want* to get these expressions to match so we can take the difference $e^{A+B} - e^A e^B$ and see what's left. In order to do that we need to “sort” the A 's and B 's by swapping, and every time we swap we will pick up a commutator.

First, it helps to rewrite (1.10) as

$$e^A e^B = \sum_{j=0}^{\infty} \frac{1}{j!} \sum_{k=0}^j \binom{j}{k} A^k B^{j-k}. \quad (1.11)$$

We leave the verification of this to the reader. The reason this is helpful is that we have a $1/j!$ leading coefficient in both terms, so we can write the difference as

$$e^{A+B} - e^A e^B = \sum_{j=0}^{\infty} \frac{1}{j!} \left[(A+B)^j - \sum_{k=0}^j \binom{j}{k} A^k B^{j-k} \right]. \quad (1.12)$$

We're nearly there, we just need to expand $(A+B)^j$. By examining the first few powers $(A+B)^0 = I$, $(A+B)^1 = A+B$, $(A+B)^2 = A^2 + AB + BA + B^2$, ..., we can see that the order of terms follows the pattern

$$(A+B)^j = \sum_{x \in \{0,1\}^j} \prod_{k=1}^j A^{\bar{x}_k} B^{x_k} \quad (1.13)$$

where x_k is the k th bit of the length j binary number x and the overhead bar $\bar{x}_k$ denotes the negation. For example, suppose $x = 00$. Then $\prod_{k=1}^j A^{\bar{x}_k} B^{x_k} = A^1 B^0 A^1 B^0 = A^2$. One can do the same for $x = 01$ to obtain $A^1 B^0 A^0 B^1 = AB$. Similarly for $x = 10$ we have $A^0 B^1 A^1 B^0$. And for $x = 11$ we have $A^0 B^1 A^0 B^1$. If we sum over all of these we indeed match the expression for $(A+B)^2$.

We can substitute (1.13) into (1.12) to obtain

$$e^{A+B} - e^A e^B = \sum_{j=0}^{\infty} \frac{1}{j!} \left[\sum_{x \in \{0,1\}^j} \prod_{k=1}^j A^{\bar{x}_k} B^{x_k} - \sum_{k=0}^j \binom{j}{k} A^k B^{j-k} \right]. \quad (1.14)$$

What a mess! I swear it's helping. To finalize our ability to subtract we just need to expand the final term $\sum_{k=0}^j \binom{j}{k} A^k B^{j-k}$ in the same binary way. That is, by changing the sum over decimal integers $\sum_{k=0}^j$ to the same binary summation $\sum_{x \in \{0,1\}^j}$. We can do this easily if we agree on the temporary (non-standard) notation that $|x|$ represents the decimal number of the binary number x . For example, $|00| = 0$, $|01| = 1$, $|10| = 2$, and $|11| = 3$. Then, we can write

$$e^{A+B} - e^A e^B = \sum_{j=0}^{\infty} \frac{1}{j!} \left[\sum_{x \in \{0,1\}^j} \left(\prod_{k=1}^j A^{\bar{x}_k} B^{x_k} \right) - A^{|x|} B^{j-|x|} \right]. \quad (1.15)$$

Astute readers may wonder what happened to the binomial coefficient $\binom{j}{k}$. The answer is it's still there — it's implicit in the binary summation and the sorted A 's and B 's. For example, consider enumerating over length $j = 2$ binary strings. We would obtain $\sum_{x \in \{0,1\}^2} A^{|x|} B^{2-|x|} = A^2 + AB + AB + B^2 = A^2 + 2AB + B^2$. We obtain the term AB *twice* — once from $x = 01$ and once from $x = 10$. This is exactly what we get from the binomial coefficient. From this one can intuit, and verify, that summing over all binary strings is equivalent to summing over decimal integers with the binomial coefficient.

Now, we want to get the unsorted terms on the left sorted like on the right. That is to say, we want all A factors before all the B factors for the term on the left. With the subtraction these terms will then disappear. How are we able to sort the terms on the left? The key is by using the commutator

$$[A, B] := AB - BA \quad (1.16)$$

which of course tells us that

$$BA = AB - [A, B]. \quad (1.17)$$

This means that, any time we see a B factor before an A factor, we can swap them, at the cost of picking up a commutator.

Example 1: Sorting a term by swapping.

Consider an example term arising in (1.15) such as $ABAB - A^2B^2$. Let's make the substitution $BA = AB - [A, B]$ to get

$$\begin{aligned} ABAB - A^2B^2 &= A(AB - [A, B])B - A^2B^2 \\ &= A^2B^2 - A[A, B]B - A^2B^2 \\ &= A[A, B]B. \end{aligned}$$

Thus, we see that sorting enable us to eliminate the A^2B^2 , and we are left with a term with a commutator, $A[A, B]B$. Now, ultimately we want to consider the size of the difference $e^{A+B} - e^A e^B$ in (1.15), so we want to take the magnitude of this difference. From above we see that

$$\|ABAB - A^2B^2\| = \|A[A, B]B\| \leq \|A\| \| [A, B] \| \|B\| \leq \| [A, B] \| \quad (1.18)$$

where in the last equality we used the assumption that $\|A\| \leq 1$ and $\|B\| \leq 1$.

This example is key to understanding how to evaluate the magnitude of the difference $e^{A+B} - e^A e^B$. From (1.15), we have

$$\|e^{A+B} - e^A e^B\| \leq \sum_{j=0}^{\infty} \frac{1}{j!} \left\| \sum_{x \in \{0,1\}^j} \left(\prod_{k=1}^j A^{\bar{x}_k} B^{x_k} \right) - A^{|x|} B^{j-|x|} \right\| \quad (1.19)$$

$$\leq \sum_{j=0}^{\infty} \frac{1}{j!} 2^j \max_{x \in \{0,1\}^j} \left\| \prod_{k=1}^j A^{\bar{x}_k} B^{x_k} - A^{|x|} B^{j-|x|} \right\|. \quad (1.20)$$

To get the second inequality, we replaced the sum over all $x \in \{0,1\}^j$ with the worst case, and multiplied by 2^j , the total number of terms in the sum. Now, we have to consider how many times we need to swap terms (by inserting a commutator) to sort $\prod_{k=1}^j A^{\bar{x}_k} B^{x_k}$. In the worst case we would have a B on the far left. How many times do we need to swap? This is a length j string, so the number of swaps would be j . In addition, because there are j total terms, we have to do j swaps at most j times. Thus, an upper bound on the number of swaps is j^2 . Every time we swap, we pick up a commutator, so we pick up at most j^2 commutators. Thus we can write

$$\|e^{A+B} - e^A e^B\| \leq \sum_{j=0}^{\infty} \frac{1}{j!} 2^j j^2 \| [A, B] \|. \quad (1.21)$$

This is a convergent sum, so at last we can see that

$$\|e^{A+B} - e^A e^B\| = O(\| [A, B] \|), \quad (1.22)$$

which is what we wanted to prove. $\square$

Lecture 2

Quantum circuits for Trotter evolution

Contents

2.1 The general idea	6
2.2 Exponentials of individual Paulis	7
2.2.1 Pauli operators and Pauli rotations	7
2.2.2 Exponentials of products of Z operators	7
2.2.3 Exponentials of general (not just Z) Paulis	8

Now that we've stated and proved Trotter formulae in the previous lecture, let's consider how to apply them. Trotter formulae give us a product of exponentials of Paulis, so the main question we have is how to do the exponential of an (n -qubit) Pauli in a quantum circuit. That is, applying the (first order) Trotter formula to $\mathcal{H} = \sum_{j=1}^N c_j P_j$ yields $\exp(-i\mathcal{H}t) \approx \prod_{j=1}^N \exp(-ic_j P_j t)$, so if we have a way to implement $\exp(-ic_j P_j t)$ in a circuit, we just apply this to every Pauli in the Hamiltonian to build a quantum circuit.

Since Paulis are hermitian, we know that $\exp(-ic_j P_j t)$ is unitary. We could just stop there, but it is useful to decompose this into unitaries which are easily implementable on quantum computers. While this varies a little between computers, a fairly universal gateset is CNOT between any two qubits and arbitrary single-qubit rotations. This is the gateset we will target for this lecture.

2.1 The general idea

For any normal operator M — i.e., an operator such that $M^\dagger M = M M^\dagger$ — we can find a diagonalization $M = V \Lambda V^\dagger$ where V is unitary and Λ is diagonal. Because

$$e^M := \sum_{k=0}^{\infty} M^k / k!, \quad (2.1)$$

it follows that

$$e^M = e^{V \Lambda V^\dagger} = \sum_{k=0}^{\infty} (V \Lambda V^\dagger)^k / k! = V \left(\sum_{k=0}^{\infty} \Lambda^k / k! \right) V^\dagger = V e^\Lambda V^\dagger. \quad (2.2)$$

In the third equality we used the fact that $(V \Lambda V^\dagger)^k = V \Lambda V^\dagger V \Lambda V^\dagger \dots V \Lambda V^\dagger = V \Lambda^k V^\dagger$, since $V^\dagger V = I$ by definition of unitary. This gives us a general framework, as long as we are able to implement V and e^Λ via CNOTs and single-qubit Pauli rotations. When M is a Pauli, V and e^Λ have particularly nice structure.

2.2 Exponentials of individual Paulis

2.2.1 Pauli operators and Pauli rotations

We recall the definition of the single-qubit Pauli operators below.

Definition 2.1 (Pauli Operators). The **Pauli operators** in the computational basis are defined by

$$\sigma_0 \equiv I \equiv \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \sigma_1 \equiv \sigma_X \equiv X \equiv \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \sigma_2 \equiv \sigma_Y \equiv Y \equiv \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}, \sigma_3 \equiv \sigma_Z \equiv Z \equiv \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}. \quad (2.3)$$

The diagonalization of these operators is given by

$$H X H = Z \quad (2.4)$$

$$(H S) Y (H S)^\dagger = Z \quad (2.5)$$

where $H := \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$ is the Hadamard gate and $S := \text{diag}(1, i)$ is the phase gate. We also recall the rotations generated by these operators.

Definition 2.2 (Pauli Rotation Gates.). The **Pauli rotation gates** are defined by

$$e^{-i\sigma_j\theta/2} = \cos(\theta/2)I + i\sin(\theta/2)\sigma_j \quad (2.6)$$

where $j = 1, 2, 3$, σ_j is a Pauli operator, and $0 \leq \theta < 2\pi$. Explicitly,

$$R_X(\theta) \equiv e^{-iX\theta/2} = \cos(\theta/2)I + i\sin(\theta/2)X = \begin{bmatrix} \cos\theta/2 & -i\sin\theta/2 \\ -i\sin\theta/2 & \cos\theta/2 \end{bmatrix}, \quad (2.7)$$

$$R_Y(\theta) \equiv e^{-iY\theta/2} = \cos(\theta/2)I + i\sin(\theta/2)Y = \begin{bmatrix} \cos\theta/2 & -\sin\theta/2 \\ \sin\theta/2 & \cos\theta/2 \end{bmatrix}, \quad (2.8)$$

$$R_Z(\theta) \equiv e^{-iZ\theta/2} = \cos(\theta/2)I + i\sin(\theta/2)Z = \begin{bmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{bmatrix}. \quad (2.9)$$

2.2.2 Exponentials of products of Z operators

In the Trotter formula we are concerned with n -qubit Pauli operators, i.e. operators of the form $\sigma^{[1]} \otimes \dots \otimes \sigma^{[n]}$ where each σ is a single-qubit Pauli. We begin with such an operator that is already diagonal, namely $Z \otimes \dots \otimes Z$ and show how to implement this in a quantum circuit. The case for a general, non-diagonal Pauli then follows from the exponentiation rule (2.2) along with the Pauli diagonalization rules (2.4) and (2.5).

It's instructive to start with $Z \otimes Z \equiv ZZ = \text{diag}(1, -1, -1, 1)$. Since this is diagonal, we can easily compute

$$e^{-iZZt} = \begin{bmatrix} e^{-it} & & & \\ & e^{it} & & \\ & & e^{it} & \\ & & & e^{-it} \end{bmatrix}. \quad (2.10)$$

Think about applying this operator to a computational basis state $|a, b\rangle \equiv |a\rangle \otimes |b\rangle$ where $a, b \in \{0, 1\}$. Inspecting the matrix above we see that the cases where $a = b$ get an e^{-it} phase (these are the upper left $a = b = 0$ and the lower right $a = b = 1$), and the cases where $a \neq b$ get an e^{it} phase (these are the inner diagonal elements $a = 0, b = 1$ and $a = 1, b = 0$). Thus we can write the action as

$$e^{-iZZt}|a, b\rangle = e^{-it(-1)^{a \oplus b}}|a, b\rangle. \quad (2.11)$$

Here, as usual, $\oplus$ denotes addition modulo 2. This $a \oplus b$ gives us some intuition for constructing a circuit to implement this operation. In particular, recall the action of the CNOT gate is

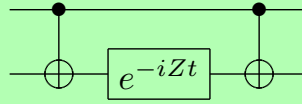
$$\text{CNOT}|a, b\rangle = |a, a \oplus b\rangle. \quad (2.12)$$

This gives us the quantity $a \oplus b$ that we're looking for. How do we get a -1 phase conditional on $a \oplus b$ now? Recall the action of the single-qubit Z rotation $e^{-iZt} = \text{diag}(e^{-it}, e^{it})$. If we apply this to the second qubit (which is in the state $|a \oplus b\rangle$), we'll thus get a phase of e^{-it} if $a \oplus b = 0$, and we'll get a phase of e^{it} if $a \oplus b = 1$. Exactly what we wanted! The only thing is that we had to modify our original computational basis state from $|a, b\rangle$ to $|a, a \oplus b\rangle$ in order to do this. It is easy to restore it to its original state, however, via the action of another CNOT:

$$\text{CNOT}|a, a \oplus b\rangle = |a, a \oplus a \oplus b\rangle = |a, b\rangle. \quad (2.13)$$

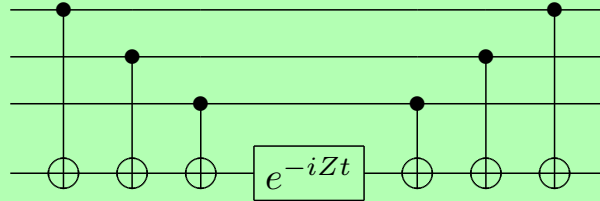
Summarizing, we have just shown the following.

Theorem 2.1. The following quantum circuit implements e^{-iZZt} .



This rule can actually be iterated and used to inductively prove the following.

Theorem 2.2 (A quantum circuit for exponentiating a product of Pauli Z s.). The following quantum circuit implements $e^{-iZ \otimes Z \otimes \dots \otimes Zt}$. Note that there are n Z products in the exponential, and the quantum circuit is drawn for the example of $n = 4$.



Note that, if some Z terms are “missing”, e.g. we want to exponentiate $ZIIZ$, then we simply remove the respective CNOTs, here the CNOTs controlled on second and third qubits for the example of $ZIIZ$.

2.2.3 Exponentials of general (not just Z) Paulis

The ability to exponentiate any string of Z operators gives us the ability to exponentiate any Pauli string through the diagonalization identities (2.4) and (2.5) and the exponentiation rule (2.2).

Example 1: Time evolution of a general Pauli

Suppose we wish to exponentiate the Pauli $XYZ \equiv X \otimes Y \otimes Z$. We can write

$$XYZ = (H \otimes HS \otimes I)(ZZZ)(H \otimes HS \otimes I)^\dagger \quad (2.14)$$

from (2.4) and (2.5). Now, from (2.2) we have that

$$e^{-iXYZt} = (H \otimes HS \otimes I)e^{-iZZZt}(H \otimes HS \otimes I)^\dagger. \quad (2.15)$$

Thus, the circuit for e^{-iXYZt} is the circuit for e^{-iZZZt} appropriately padded with single-qubit rotations.

Lecture 3

Coding exercise: Trotterized time evolution for the transverse-field Ising model

[Self-link to this Colab](#), in case you are viewing the PDF.

3.1 Overview

Time evolution refers to (computationally) solving for the dynamics of a quantum system given an initial state and Hamiltonian. From the Schrodinger equation for (time-independent) Hamiltonian H we can immediately write down the solution as $\exp(-iHt)$ applied to the initial state. But how do you actually evaluate $\exp(-iHt)$? As this is a central question many techniques and approximations have been developed. You will implement one important and widely applicable approximation known as Trotterization (or the Trotter formula) to do this.

3.1.1 Setup

Install packages here. You are free to use any quantum information software package like [Cirq](#), [Qiskit](#), or [QuTiP](#). For the exact numerics, use NumPy and SciPy.

```
[ ]: !pip install cirq-core qiskit qiskit_aer --quiet
```

```
[ ]: import itertools
import functools

import matplotlib.pyplot as plt
import networkx as nx
import numpy as np
import scipy
import tqdm

# If you want to use good software.
import cirq

# If you want to use a cloud quantum computer (or want to use bad software and/or like ↵
↵ misery).
import qiskit
import qiskit_aer
```

```
[ ]: """[Unimportant] Set notebook options."""
import warnings
warnings.filterwarnings("ignore", category=tqdm.TqdmDeprecationWarning)

plt.rcParams.update({"font.family": "serif", "font.size": 16})
plot_options = {"cmap": "bwr", "vmin": -1.0, "vmax": 1.0, "shading": "nearest"}
```

3.2 The transverse-field Ising model Hamiltonian

We will consider the one-dimensional transverse-field Ising model (TFIM) on n sites (qubits) with periodic boundary conditions

$$H = J \sum_{i=1}^n Z_i Z_{i+1} - \Gamma \sum_{i=1}^n X_i.$$

Here X and Z are Pauli operators which in the computational basis have matrices

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}.$$

Subscripts refer to the sites (qubits) the Pauli acts on, and products are tensor products. So, for example, $Z_1 Z_2 \equiv Z_1 \otimes Z_2 \equiv Z \otimes Z \otimes I$ with $n = 3$.

Exact calculations

Before we start with approximations, let's do some classical brute-force, exact (unscalable) calculations.

3.2.1 A function for the TFIM Hamiltonian

Write a function to return the TFIM Hamiltonian H as a NumPy array for a given n , J , and Γ .

```
[ ]: def get_tfim_hamiltonian(
    nqubits: int,
    coupling: float = 1,
    gamma: float = 0.25,
) -> np.ndarray:
    qubits = circ.LineQubit.range(nqubits)

    pauli_sum = coupling * sum(
        circ.Z.on(qubits[i]) * circ.Z.on(qubits[(i + 1) % nqubits]) for i in
        range(nqubits)
    ) - gamma * sum(circ.X.on(qubits[i]) for i in range(nqubits))

    return pauli_sum.matrix()
```

```
[ ]: get_tfim_hamiltonian(nqubits=3).real
```

```
[ ]: array([[ 3.  , -0.25, -0.25,  0.  , -0.25,  0.  ,  0.  ,  0.  ],
          [-0.25, -1.  ,  0.  , -0.25,  0.  , -0.25,  0.  ,  0.  ],
          [-0.25,  0.  , -1.  , -0.25,  0.  ,  0.  , -0.25,  0.  ],
          [ 0.  , -0.25, -0.25, -1.  ,  0.  ,  0.  ,  0.  , -0.25],
          [-0.25,  0.  ,  0.  ,  0.  , -1.  , -0.25, -0.25,  0.  ],
          [ 0.  , -0.25,  0.  ,  0.  , -0.25, -1.  ,  0.  , -0.25],
          [ 0.  ,  0.  , -0.25,  0.  , -0.25,  0.  , -1.  , -0.25],
          [ 0.  ,  0.  ,  0.  , -0.25,  0.  , -0.25, -0.25,  3.  ]])
```

Note: The Hamiltonian is sparse and can be more compactly represented with objects in `scipy.sparse`. Here we just use (dense) NumPy arrays for simplicity with the small ($n = 10$) problem size.

For the remaining problems, use the TFIM Hamiltonian H with $n = 10$, $J = 1$, and $\Gamma = 0.25$.

3.3 Exact time evolution

Compute and plot the magnetization $\langle Z_j \rangle(t) = \langle \psi(t) | Z_j | \psi(t) \rangle$ for each site (qubit) $j = 1, \dots, 10$ and each time t in `np.linspace(0, 50, 100)`.

- Here, $|\psi(t)\rangle = \exp(-iHt)|\psi(0)\rangle$
- Use $|\psi(0)\rangle = |0000100000\rangle$ for the initial state. (All spins down except for one spin up in the middle.)

For plotting, use `matplotlib.pyplot.imshow` or `matplotlib.pyplot.pcolormesh` or similar.

```
[ ]: """Set parameters for Hamiltonian."""
nqubits = 8
coupling = 1.0
gamma = 0.25

ham = get_tfim_hamiltonian(nqubits=nqubits, coupling=coupling, gamma=gamma)

[ ]: """Set parameters for initial state."""
qubits = cirq.LineQubit.range(nqubits)
upqubit = nqubits // 2 - 1

[ ]: """Set parameters for time evolution."""
tmin = 0.0
tmax = 50.0
tsteps = 100

times = np.linspace(tmin, tmax, tsteps)

[ ]: """Compute magnetizations via exact time evolution."""
# Allocate array to store the magnetizations.
magnetizations = np.empty(shape=(tsteps, nqubits))

# Set the initial state.
istates = [cirq.KET_ZERO.state_vector()] * nqubits
istates[upqubit] = cirq.KET_ONE.state_vector()
istate = functools.reduce(np.kron, istates)

# Do the time evolution.
for t in tqdm.tqdm_notebook(range(tsteps)):
    # Compute the time-evolved state. This is what we'll use Trotterization for in the
    # next problem!
    psi = scipy.linalg.expm(-1j * ham * times[t]) @ istate

    # Compute the magnetization per site.
    for j in range(nqubits):
        observable = cirq.Z.on(qubits[j]).matrix(qubits)
        magnetization = np.real_if_close(psi.conj().T @ observable @ psi)
        magnetizations[t, j] = magnetization
```

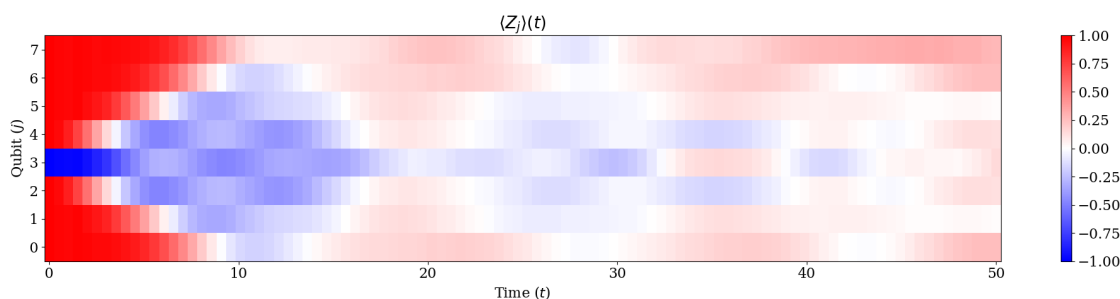
0% | 0/100 [00:00<?, ?it/s]

```
[ ]: """Plot magnetizations."""
plt.figure(figsize=(20, 5))

plt.pcolormesh(times, range(nqubits), magnetizations.T, **plot_options)

plt.xlabel(r"Time ($t$)")
plt.ylabel(r"Qubit ($j$)")
plt.title(r"$\langle Z_j \rangle$ vs $t$")

plt.colorbar()
plt.tight_layout();
```



3.4 Trotterized time evolution

Consider a Hamiltonian $H = H_A + H_B$. If $[H_A, H_B] = 0$ (square brackets denote commutation), exponentiation is very simple: $\exp(-iHt) = \exp(-iH_A t) \exp(-iH_B t)$.

Of course $[H_A, H_B] \neq 0$ in general. The [first order \(Lie-\) Trotter \(-Suzuki\) formula](#) says that, if $\|H_A\|, \|H_B\| \leq 1$,

$$\exp(-i(H_A + H_B)t) = \exp(-iH_A t) \exp(-iH_B t) + O(\|H_A, H_B\|t^2).$$

For the TFIM we can split our Hamiltonian into two mutually commuting groups, namely $H_Z := J \sum_{i=1}^n Z_i Z_{i+1}$ and $H_X := -\Gamma \sum_{i=1}^n X_i$, and apply this formula with $H = H_Z + H_X$.

3.4.1 A function for 1st order Trotter

Write a function to input time t and return the (quantum circuit) operations for the first-order Trotter formula with the TFIM. Use the same n , J , and Γ as above.

Using Cirq

```
[ ]: # TODO: Complete either the _unitary_ or the _decompose_ method in the custom gate_
      below,
      # then complete the trotter_step function.
class ExpZZ(cirq.Gate):
    def __init__(self, theta):
        super(ExpZZ, self)
        self.theta = theta
```

```

def _num_qubits_(self):
    return 2

# Use the unitary...
def _unitary_(self):
    pass # TODO: Your code here.

# ...or use a decomposition.
# def _decompose_(self, qubits):
#     a, b = qubits
#     yield cirq.CNOT(a, b)
#     yield ...

def _circuit_diagram_info_(self, args):
    return f"ExpZZ({self.theta})"

def trotter_step(time: float) -> cirq.OP_TREE:
    pass # Your code here.

```

Using Qiskit

```

[ ]: # If you're using Qiskit, write a function to do this task here. An optional template
    ↪ is provided.
def trotter_step_qiskit(
    circuit_qiskit: qiskit.QuantumCircuit,
    qreg: qiskit.QuantumRegister,
    time: float,
    add_barriers: bool = True,
) -> None:
    pass # Your code here.

```

3.4.2 Simulation to compute the magnetization per qubit

Simulate your circuit using 10^4 shots (samples) to compute the magnetization per qubit using the same parameters as above, and plot the result.

```

[ ]: # Set parameters. Note: `ntrotter` introduced in problem (1.5).
nshots = 10_000
ntrotter = 1

```

Using Cirq *Note:* This cell assumes some variables, e.g. that you kept the name `trotter_step` from the template function. Update as needed.

```

[ ]: # Allocate array to store magnetizations.
magnetizations_trotter = np.empty(shape=(tsteps, nqubits))

for t in tqdm.tqdm_notebook(range(tsteps)):
    # Get the circuit to do 1st order Trotter.
    circuit = cirq.Circuit(
        # Initial state.
        cirq.Moment(cirq.X.on(qubits[upqubit])),
        # Trotter step(s).
    )

```

```

    [trotter_step(times[t] / ntrotter) for _ in range(ntrotter)],
    # Measurements.
    cirq.Moment(cirq.measure(qubits[i], key=str(i)) for i in range(nqubits)),
)

# Sample from the circuit.
result = cirq.Simulator().run(circuit, repetitions=nshots)

# Compute and store magnetizations.
for j in range(nqubits):
    measurements = result.histogram(key=str(j))
    magnetization = (measurements.get(0, 0.0) - measurements.get(1, 0.0)) / nshots
    magnetizations_trotter[t, j] = magnetization

```

```

[ ]: """Plot the magnetizations."""
plt.figure(figsize=(20, 5))

plt.pcolormesh(times, range(nqubits), magnetizations_trotter.T, **plot_options)

plt.xlabel(r"Time ($t$)")
plt.ylabel(r"Qubit ($j$)")
plt.title(rf"$\langle Z_j \rangle$ via {ntrotter} 1st order Trotter step(s)")

plt.colorbar()
plt.tight_layout();

```

Using Qiskit

```

[ ]: # Your code here.

```

3.4.3 Increasing the number of Trotter steps

You should see that the previous result is poor! (Decent for the first few time steps, but very inaccurate after.) This can be improved by implementing a number r of Trotter steps, essentially repeating the above circuit r times where each Trotter step is done for time t/r . Specifically, the first-order Trotter formula with r steps says that

$$\exp(-i(H_A + H_B)t) = [\exp(-iH_A t/r) \exp(-iH_B t/r)]^r + O\left(\frac{\| [H_A, H_B] \| t^2}{r}\right).$$

This is now nice because the error goes to zero as $r \rightarrow \infty$, and we can decrease the error by using more steps. The cost, of course, is more gates in the circuit.

Do the same simulation as (1.4) but now using $r = 10, 20$, and 40 Trotter steps. Plot the results and compare to the exact solution.

```

[ ]: # Set parameters.
nshots = 10_000
ntrotter = 40

# Allocate array to store magnetizations.
magnetizations_trotter = np.empty(shape=(tsteps, nqubits))

for t in tqdm.notebook.tqdm(range(tsteps)):
    # Get the circuit to do 1st order Trotter.

```

```

circuit = cirq.Circuit(
    # Initial state.
    cirq.Moment(cirq.X.on(qubits[upqubit])),
    # Trotter step(s).
    [trotter_step(times[t] / ntrotter) for _ in range(ntrotter)],
    # Measurements.
    cirq.Moment(cirq.measure(qubits[i], key=str(i)) for i in range(nqubits)),
)

# Sample from the circuit.
result = cirq.Simulator().run(circuit, repetitions=nshots)

# Compute and store magnetizations.
for j in range(nqubits):
    measurements = result.histogram(key=str(j))
    magnetization = (measurements.get(0, 0.0) - measurements.get(1, 0.0)) / nshots
    magnetizations_trotter[t, j] = magnetization

```

```

[ ]: """Plot the magnetizations."""
plt.figure(figsize=(20, 5))

plt.pcolormesh(times, range(nqubits), magnetizations_trotter.T, **plot_options)

plt.xlabel(r"Time ($t$)")
plt.ylabel(r"Qubit ($j$)")
plt.title(rf"$\langle Z_j \rangle$ via {ntrotter} 1st order Trotter step(s)")

plt.colorbar()
plt.tight_layout();

```

3.5 Further reading and exercises

- Re-implement this exercise with another Hamiltonian, e.g. the Lipkin model.
- Compute expectation values from the final time-evolved state, e.g. the energy $\langle \psi(t) | H | \psi(t) \rangle$.
- We looked at the first-order Trotter formula, but there are higher orders: Read more, e.g., [here](#) and in the [Theory of Trotter error](#).
- You may be interested in [imaginary time evolution](#), another technique to find the ground state of a time-independent Hamiltonian.
- Want to simulate a time-dependent Hamiltonian? You may want to know about the [quantum adiabatic theorem](#).

If you are an instructor and wish to see solutions, email rmlarose@msu.edu.

Bibliography

- [CST⁺21] Andrew M. Childs, Yuan Su, Minh C. Tran, Nathan Wiebe, and Shuchen Zhu. Theory of trotter error with commutator scaling. *Physical Review X*, 11(1):011020, February 2021.
- [FICS24] Roland C. Farrell, Marc Illa, Anthony N. Ciavarella, and Martin J. Savage. Quantum simulations of hadron dynamics in the schwinger model using 112 qubits. (arXiv:2401.08044), June 2024. arXiv:2401.08044 [quant-ph].

